



МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ –
МСХА имени К. А. ТИМИРЯЗЕВА»
(ФГБОУ ВО РГАУ – МСХА имени К. А. Тимирязева)

Институт экономики и управления
Кафедра статистики и кибернетики

Методы и средства проектирования информационных систем

КУРСОВОЙ ПРОЕКТ

на тему: «Проектирование и разработка базы данных для веб-сервиса
обмена книгами».

Выполнила:
Студентка 3 курса группы Д-Э311
Чуркина М.И.

Дата регистрации КП
на кафедре: _____

Допущена к защите
Руководитель: _____

учёная степень, учёное звание, ФИО

Члены комиссии:

учёная степень, учёное звание, ФИО _____
подпись

учёная степень, учёное звание, ФИО _____
подпись

учёная степень, учёное звание, ФИО _____
подпись

Оценка _____

Дата защиты _____

Москва, 2026

Содержание

Введение	3
Глава 1. Теоретическая часть	5
1.1. Понятие и классификация информационных систем	5
1.2. Методологии проектирования информационных систем .	7
1.3. Нотации моделирования	9
1.3.1. IDEF0	10
1.3.2. DFD	11
1.3.3. BPMN	12
1.3.4. Элементы UML	14
1.3.5. ER-диаграмма	15
1.4. Базы данных: понятие, классификация и место в ИС	16
1.5. Реляционные базы данных и их проектирование	18
1.6. Выбор СУБД	20
Глава 2. Практическая часть	22
2.1. Описание предметной области	22
2.2. ER-диаграмма базы данных	24
2.3. Структура таблиц и ограничения целостности	25
2.4. DDL-скрипты создания базы данных	35
2.5. Индексы	37
2.6. Примеры запросов	39
Заключение	43
Библиографический список	45
Приложение	47

Введение

Информационные системы стали неотъемлемой частью повседневной жизни современного человека. Они проникают во все сферы деятельности: от покупки товаров до поиска попутчиков, от онлайн-образования до организации совместных поездок. Особое место среди них занимают шеринговые платформы – сервисы, позволяющие людям делиться вещами, услугами или ресурсами без посредников. Экономика совместного потребления активно развивается: всё больше людей предпочитают не покупать новые вещи, а брать их во временное пользование или обменивать.

Одной из сфер, где шеринговые модели особенно востребованы, является обмен книгами. Ежегодно издаются миллионы экземпляров, многие из которых после прочтения остаются на полках пылиться или отправляются в макулатуру. При этом существует огромное количество людей, которые хотели бы прочитать ту или иную книгу, но не готовы покупать её из-за высокой стоимости или отсутствия места в доме. Традиционные способы обмена – буккроссинг, библиотеки, объявления в социальных сетях – имеют существенные ограничения: отсутствие гарантий надёжности участников, сложность поиска конкретного издания, отсутствие системы рейтингов и учёта истории обменов.

Возникает закономерный вопрос: как создать удобный, безопасный и прозрачный сервис, который позволит пользователям находить друг друга, обмениваться книгами и доверять участникам сделки? Решение этой проблемы лежит в области проектирования информационных систем, а именно база данных обеспечивает хранение информации о пользователях, книгах, геолокации, запросах на обмен, отзывах и рейтингах. Без правильно спроектированной и оптимизированной базы данных невозможна ни быстрая работа геолокационного поиска, ни целостность запросов на обмен, ни достоверность рейтинговой системы.

Целью данного курсового проекта является проектирование и разработка базы данных для веб-сервиса обмена книгами между пользователями «BookSwar», обеспечивающей поиск, обмен и учёт книг на основе

геолокации и системы рейтингов.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1. Рассмотреть теоретические основы проектирования информационных систем.
2. Изучить классификацию баз данных по модели данных и обосновать выбор реляционной модели для разрабатываемого сервиса.
3. Спроектировать логическую структуру базы данных: выделить сущности, атрибуты, связи, привести схему к третьей нормальной форме.
4. Реализовать базу данных в PostgreSQL.
5. Создать систему индексов для оптимизации производительности запросов.
6. Разработать и протестировать ключевые запросы, охватывающие основные сценарии работы сервиса.

Актуальность данной работы обусловлена тем, что существующие аналоги (буккроссинг, Авито, Telegram-чаты, PaperBack Swap) не предоставляют полного набора функций: геолокационного поиска, рейтинга участников, автоматизированного учёта обменов и wishlist с уведомлениями. При этом потребность в удобном сервисе для обмена книгами остаётся высокой. Разработка информационной системы «BookSwap» призвана устранить эти недостатки.

В работе рассматриваются ключевые концепции проектирования реляционных баз данных: выделение сущностей и связей, нормализация до третьей нормальной формы, выбор типов данных, создание индексов для оптимизации запросов, реализация пространственных запросов с использованием PostGIS, а также обеспечение ссылочной целостности и транзакционной безопасности при обработке запросов на обмен.

Глава 1. Теоретическая часть

1.1 Понятие и классификация информационных систем

В современном мире информационные технологии играют ключевую роль в управлении бизнес-процессами, хранении данных и автоматизации деятельности предприятий. Центральное место в этой сфере занимает понятие информационной системы. Для того чтобы грамотно спроектировать такую систему, необходимо чётко понимать её сущность, структуру и место в существующей классификации. Это позволит обоснованно выбирать методы и средства проектирования на каждом этапе разработки.

Под информационной системой принято понимать взаимосвязанную совокупность средств, методов и персонала, используемую для хранения, обработки, поиска и выдачи информации в интересах достижения поставленной цели [8]. Иными словами, информационная система – это не просто программа или база данных, а сложный комплекс, включающий в себя аппаратное обеспечение, программное обеспечение, данные, людей, которые с ними работают, а также правила и инструкции, регламентирующие эту работу.

Важно подчеркнуть, что основное назначение любой информационной системы – обеспечение пользователя необходимой информацией для принятия решений [5]. Поэтому при проектировании информационной системы всегда нужно отвечать на вопрос: какие задачи пользователя она решает и какую информацию предоставляет. Это отличает информационную систему от просто набора программ или файлов на компьютере.

Информационные системы можно классифицировать по разным признакам. Наиболее значимыми с точки зрения проектирования являются классификации по масштабу, по архитектуре, по степени автоматизации и по предметной области [10].

По масштабу информационные системы делятся три подгруппы:

1) Персональные. Эти системы предназначены для работы одного пользователя, например, личный органайзер, домашняя бухгалтерия

или планер.

2) Групповые. Такие системы поддерживают работу небольшого коллектива, обычно до нескольких десятков человек, и часто функционируют в пределах локальной сети.

3) Корпоративные. Эти системы охватывают деятельность целой организации, могут насчитывать тысячи пользователей и работать на значительных территориях.

По архитектурному принципу выделяют так же три вида:

1) Локальные. Эти системы целиком работают на одном компьютере.

2) Распределённые. Такие системы предполагают, что разные компоненты находятся на разных машинах (устройствах), объединенных сетью. При этом часто используется клиент-серверная архитектура, где на сервере хранятся данные, а на клиентских компьютерах выполняется их обработка и отображение [10].

3) Многоуровневые системы. При такой архитектуре в систему добавляют между клиентом и сервером базы данных промежуточное звено – сервер приложений, который берёт на себя часть вычислительной нагрузки.

По степени автоматизации информационные системы подразделяются еще на три группы:

1) Ручные. Эти системы предполагают, что все операции по обработке информации выполняются человеком самостоятельно.

2) Автоматизированные. Такие системы сочетают в себе участие человека и компьютера: человек вводит данные и принимает решения, а компьютер выполняет вычисления, сортировку, поиск и другие рутинные операции.

3) Автоматические. Это системы, которые полностью работают без участия человека по заранее заданным алгоритмам.

По предметной области и характеру решаемых задач выделяют три вида:

1) Транзакционные системы. Иначе называются OnLine Transaction Processing (OLTP), и предназначены для регистрации и обработки большого количества однотипных операций в реальном времени – примером может служить банковская система обработки платежей.

2) Аналитические системы. Иначе - OnLine Analytical Processing (OLAP), и служат для анализа больших объёмов данных и поддержки принятия сложных управленческих решений.

3) Системы электронной коммерции. Такие системы обеспечивают куплю-продажу товаров и услуг через интернет [2].

Проектируемый в рамках данной курсовой работы веб-сервис обмена книгами «BookSwap» можно классифицировать следующим образом: по масштабу его можно отнести к групповым или малым корпоративным системам, поскольку он рассчитан на неограниченное количество пользователей, но не привязан к конкретной организации; По архитектуре это типичная многоуровневая веб-ориентированная система, где клиент - браузер пользователя, сервер приложений реализован на фреймворке FastAPI, а сервер базы данных – PostgreSQL; По степени автоматизации «BookSwap» является автоматизированной системой, так как ключевые действия инициируются пользователем, а система автоматически обрабатывает запросы, выполняет поиск и отправляет уведомления; По типу решаемых задач это, в первую очередь, транзакционная система, поскольку основную нагрузку создают операции добавления книг, создания запросов на обмен и подтверждения сделок. При этом система также содержит элементы аналитики - например, расчёт среднего рейтинга пользователя на основе всех полученных им отзывов.

1.2 Методологии проектирования информационных систем

Проектирование информационной системы представляет собой сложный многоэтапный процесс, требующий выбора определённого порядка действий, набора методов и инструментов. Совокупность этих принципов и подходов называется методологией проектирования [19]. От правильного выбора методологии напрямую зависят сроки разработки, качество конечного продукта и удобство сопровождения системы в будущем.

Все существующие методологии проектирования информационных систем можно разделить на две большие группы: классические (каскад-

ные) и гибкие (итеративные).

Каскадная модель была предложена Уинстоном Ройсом в 1970 году и долгое время оставалась основным стандартом в разработке программного обеспечения. Данная модель предполагает последовательное выполнение этапов, где каждый следующий этап начинается только после полного завершения предыдущего. Типичный каскадный процесс включает следующие стадии: анализ требований, проектирование, реализация, тестирование и внедрение.

Основным преимуществом каскадной модели является чёткая документация на каждом этапе и простота управления проектом. Кроме того, последовательный подход позволяет точно оценить сроки и бюджеты на начальных стадиях [4]. Однако у этой методологии есть и существенные недостатки: невозможность вернуться на предыдущий этап без значительных затрат, а также требование полной определённости всех требований в самом начале проекта. На практике это означает, что ошибки, обнаруженные на этапе тестирования, могут потребовать изменений во всей системе.

В противовес жёсткому каскадному подходу в начале 2000-х годов был сформулирован Манифест Agile, провозгласивший новые принципы разработки программного обеспечения [17]. Ключевыми идеями Agile являются:

- Приоритет работающего продукта перед исчерпывающей документацией,
- Постоянное взаимодействие с заказчиком,
- Готовность к изменениям требований в процессе разработки,
- Частые поставки новых версий продукта [19].

Одной из наиболее распространённых реализаций Agile является Scrum. Эта методология предполагает разбиение всего процесса разработки на короткие итерации – спринты, обычно длительностью от одной до четырёх недель. В рамках каждого спринта команда создаёт работающий инкремент продукта, который может быть продемонстрирован заказчику. В Scrum выделяются три ключевые роли: владелец продукта (Product Owner), Scrum-мастер и команда разработки.

Другой популярной реализацией Agile является Kanban. В отличие от

Scrum, Kanban не использует фиксированные итерации и основан на визуализации потока задач с помощью досок. Основным принципом Kanban – ограничение количества задач, находящихся одновременно в работе, что позволяет избежать перегрузки команды и ускорить выполнение задач.

Преимуществами гибких методологий являются адаптивность к изменениям, раннее получение работающего продукта и высокая вовлечённость заказчика. Недостатками можно считать меньшую предсказуемость сроков и бюджетов, а также требование высокой дисциплины от команды разработчиков.

На практике многие организации используют смешанные подходы, комбинируя элементы каскадной модели для этапов анализа и проектирования с гибкими методами на этапах разработки и тестирования. Такая комбинация позволяет получить преимущества обеих методологий, частично компенсируя их недостатки [11].

Однако, независимо от выбранной методологии – будь то строгая каскадная модель или гибкие итеративные подходы – на этапе анализа и проектирования возникает одна и та же задача: необходимо зафиксировать структуру и поведение будущей системы в виде, понятном и разработчикам, и заказчику. Решение этой задачи невозможно без использования формализованных графических языков и нотаций. Поэтому после рассмотрения методологий проектирования логично перейти к изучению основных нотаций моделирования, применяемых для описания информационных систем.

1.3 Нотации моделирования

Проектирование информационных систем – это сложный процесс, в ходе которого необходимо зафиксировать требования к будущей системе, описать её структуру, функции, потоки данных и сценарии взаимодействия пользователей. На ранних этапах разработки, когда ещё нет работающего программного кода, единственным способом зафиксировать и передать понимание системы между участниками проекта являются её графические модели. Для того чтобы такие модели были

однозначно понятны всем участникам, необходимо использовать формализованные языки и нотации.

Нотация представляет собой набор правил и графических обозначений (символов), с помощью которых можно построить модель системы. Без применения таких стандартизированных нотаций разработка сложных систем становится крайне затруднительной. Разные специалисты будут по-разному интерпретировать одни и те же схемы, что приведёт к ошибкам, недопониманию и, в конечном счёте, к несоответствию реализованной системы исходным требованиям [13].

В мировой практике сложилось несколько стандартизированных нотаций, каждая из которых решает свои специфические задачи. Одни нотации ориентированы на описание функций системы, другие – на описание потоков данных, третьи – на моделирование бизнес-процессов с учётом логики ветвлений и параллелизма. Наиболее распространёнными и востребованными в современной практике проектирования информационных систем являются IDEF0, DFD, BPMN и язык UML [16].

1.3.1 IDEF0

Нотация IDEF0 (Integration Definition for Function Modeling) была разработана в США в рамках программы автоматизации промышленных предприятий ICAM (Integrated Computer Aided Manufacturing) и впоследствии принята в качестве федерального стандарта США. Она предназначена для функционального моделирования систем, то есть для описания того, какие функции выполняет система и как они связаны между собой [9].

Основной структурной единицей IDEF0 является функциональный блок. Каждый блок представляет собой конкретную функцию или процесс, имеющий чётко определённую цель. Внутри блока указывается его название, описывающее действие.

Каждый функциональный блок в IDEF0 имеет четыре типа стрелок, которые описывают взаимодействие блока с окружающей средой и другими блоками:

Вход – это то, что преобразуется функцией. Вход может быть мате-

риальным объектом (деталь, документ) или информацией (данные, заявка).

Выход – это результат выполнения функции. Выход показывает, во что преобразуется вход после выполнения процесса.

Управление – это правила, ограничения, стандарты или инструкции, которыми руководствуется функция при своём выполнении. Управление показывает, как именно должна выполняться функция.

Механизм – это ресурсы, с помощью которых функция выполняется. Механизмами могут быть люди, оборудование, программное обеспечение или любые другие исполнители [16].

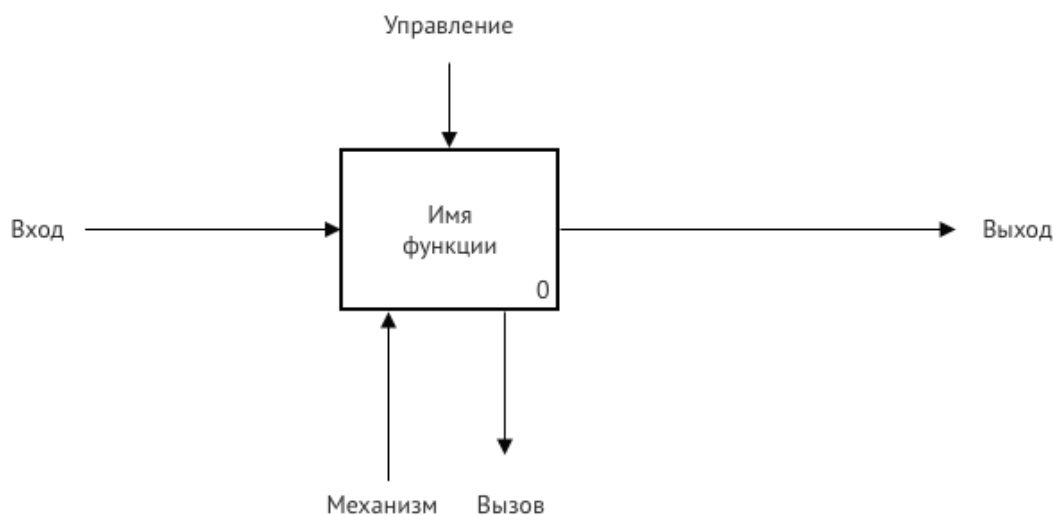


Рисунок 1 – диаграмма в нотации IDEF0

Таким образом, IDEF0 позволяет получить формальное описание функций системы и выявить ключевые сущности предметной области, что является первым шагом к проектированию структуры базы данных.

1.3.2 DFD

Нотация DFD (Data Flow Diagrams) – диаграммы потоков данных – предназначена для моделирования движения информации в системе. DFD фокусируется на том, какие данные циркулируют в системе, откуда они поступают, где хранятся и кому передаются [9].

Основными элементами DFD являются внешние сущности, процес-

сы, хранилища данных и потоки данных.

Внешняя сущность – это объект, находящийся за пределами рассматриваемой системы, который является источником или приёмником данных. Внешняя сущность взаимодействует с системой, но не управляется ею.

Процесс – это преобразователь входных потоков данных в выходные. Каждый процесс выполняет определённое действие над данными: вычисляет, сортирует, фильтрует, объединяет, изменяет. [9].

Хранилище данных – это место, где данные сохраняются между процессами. Хранилищем может быть база данных, файл, оперативный регистр или любая другая структура, обеспечивающая долговременное или временное хранение информации [20].

Поток данных – это стрелка, показывающая движение данных от одного элемента диаграммы к другому. Поток данных всегда имеет название, которое описывает передаваемую информацию. Поток может соединять внешнюю сущность с процессом, процесс с хранилищем, процесс с процессом или хранилище с процессом. Потоки не могут соединять две внешние сущности или два хранилища напрямую – только через процесс.

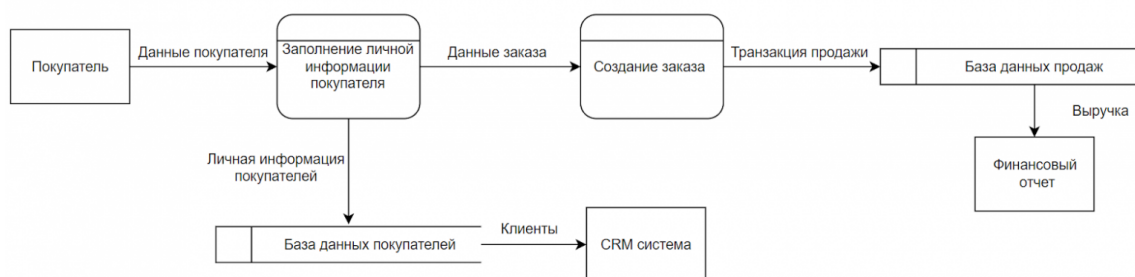


Рисунок 2 – Диаграмма в нотации DFD

Таким образом, DFD детализирует информационные потоки и уточняет структуру хранилищ данных, подготавливая почву для перехода к логическому проектированию базы данных.

1.3.3 BPMN

Нотация BPMN (Business Process Model and Notation) – это современный стандарт моделирования бизнес-процессов, который активно

используется как в бизнес-анализе, так и при проектировании информационных систем. BPMN была разработана организацией OMG (Object Management Group) и в настоящее время является общепризнанным языком для описания процессов как понятных бизнес-пользователям, так и достаточных для технической реализации.

BPMN ориентирована на динамику – последовательность выполнения действий, условия ветвления, параллельные процессы и взаимодействие между разными участниками [16].

Основными элементами BPMN являются события, задачи, шлюзы, потоки управления, а также пулы и дорожки.

События – это то, что происходит в процессе. Событие обозначается кругом и может быть начальным, промежуточным и конечным.

Задачи – это элементарные действия, которые выполняются в рамках процесса. Задача изображается скруглённым прямоугольником и содержит краткое описание действия.

Шлюзы – это точки ветвления и объединения потоков управления. Шлюз изображается ромбом и определяет логику дальнейшего выполнения процесса.

Потоки управления – это стрелки, соединяющие элементы диаграммы и показывающие последовательность выполнения.

Пулы и дорожки – это контейнеры, которые разделяют ответственность между разными участниками процесса. Пул представляет собой крупного участника. Внутри пула могут быть дорожки, которые соответствуют конкретным ролям или исполнителям. Дорожки визуально разделяют диаграмму на горизонтальные зоны, и каждая задача помещается в ту дорожку, к чьей ответственности она относится [20].

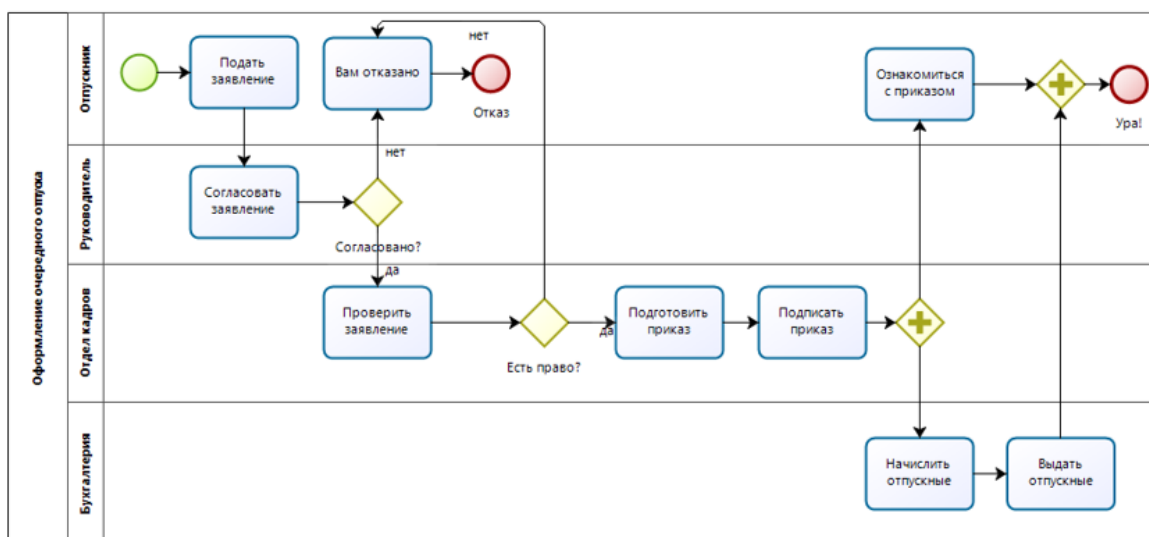


Рисунок 3 – Диаграмма в нотации BPMN

BPMN также позволяет моделировать временные события – ожидание подтверждения, тайм-ауты, напоминания. Это важно для систем с человеческим участием, где ответ может быть не мгновенным.

Таким образом, BPMN позволяет описать жизненный цикл сущностей и определить все возможные переходы между состояниями, что является важной частью логического проектирования базы данных.

1.3.4 Элементы UML

UML (Unified Modeling Language) – это унифицированный язык визуального моделирования, который широко используется при объектно-ориентированном проектировании программных систем. UML был разработан в середине 1990-х годов объединением ведущих специалистов в области объектно-ориентированного анализа и в настоящее время поддерживается консорциумом OMG [12].

UML включает в себя 14 типов диаграмм, которые принято делить на две большие группы: структурные диаграммы, описывающие статическую структуру системы, и поведенческие диаграммы, которые описывают динамику, то есть поведение системы во времени.

Как пример структурных диаграмм можно рассмотреть две, особенно важные:

Диаграмма классов является основной структурной диаграммой UML. Она описывает типы объектов (классы), их атрибуты, методы

и связи между классами. Диаграмма классов показывает, из каких элементов состоит система и как эти элементы связаны друг с другом.

Диаграмма компонентов показывает, как система физически разделена на компоненты и какие зависимости существуют между ними. Эта диаграмма полезна для понимания архитектуры системы на уровне её составных частей.

Среди поведенческих можно выделить так же две диаграммы:

Диаграмма состояний описывает жизненный цикл одного объекта: все возможные состояния, в которых он может находиться, и переходы между ними под воздействием различных событий. Эта диаграмма позволяет наглядно представить поведение объекта во времени.

Диаграмма последовательности показывает взаимодействие объектов во времени. Она отображает последовательность сообщений, которыми обмениваются объекты при выполнении определённого сценария, и временной порядок этих сообщений [1].

1.3.5 ER-диаграмма

ER-диаграмма (Entity-Relationship diagram) – это нотация, предназначенная специально для концептуального проектирования баз данных. Она была предложена Питером Ченом в 1976 году и с тех пор является стандартным инструментом для описания структур данных независимо от конкретной СУБД [10].

Основными понятиями ER-модели являются сущность, атрибут и связь.

Сущность – это объект предметной области, о котором необходимо хранить информацию. Сущность может быть реальным объектом или абстрактным понятием.

Атрибут – это характеристика сущности. Например, имя, электронная почта, пароль для сущности «Пользователь». Каждый атрибут имеет тип данных.

Связь – это отношение между двумя или более сущностями. Связи характеризуются типом (один-к-одному, один-ко-многим, многие-ко-многим) и могут быть обязательными или необязательными [18].

Ключевым понятием ER-модели является первичный ключ – атрибут или набор атрибутов, однозначно идентифицирующий каждый экземпляр сущности.

ER-диаграмма является концептуальной моделью данных – она описывает, какие данные будут храниться в системе, но не привязана к конкретной СУБД. После построения ER-диаграммы она преобразуется в логическую модель и затем в физическую.

Таким образом, все рассмотренные нотации позволяют описать систему с разных сторон, однако для непосредственного проектирования структуры хранения данных ключевое значение имеет ER-диаграмма. Именно она даёт наглядное представление о том, какие сущности будут храниться в базе данных, какими атрибутами они обладают и как связаны между собой. Поскольку данная курсовая работа нацелена на проектирование и разработку именно базы данных для веб-сервиса обмена книгами, ER-диаграмма станет основой для всех последующих этапов. Прежде чем перейти к построению ER-диаграммы для конкретной предметной области, необходимо детально рассмотреть, что такое базы данных, как они классифицируются и какое место занимают в современных информационных системах.

1.4 Базы данных: понятие, классификация и место в ИС

База данных (БД) – это совокупность данных, организованных по определённым правилам, предусматривающим общие принципы описания, хранения и манипулирования данными, независимая от прикладных программ. Эти данные относятся к определённой предметной области и организованы таким образом, что могут быть использованы для решения многих задач многими пользователями [10].

Важным моментом является то, что данные, из которых состоит база данных, должны быть тщательно и логично организованы. Поэтому БД строится по определённым правилам и должна удовлетворять ряду требований, из которых основные: минимальная избыточность, возможность актуализации, обеспечение целостности данных и возможность

обеспечения разнообразных запросов пользователей.

Для работы с базами данных создана система управления базами данных (СУБД) – комплекс программных и языковых средств, который отвечает за хранение и управление информацией. Основные функции СУБД: создание баз данных, изменение, удаление и объединение их по определённым признакам; хранение данных, в том числе больших массивов, в структурированном виде и нужном формате; защита данных от взлома и нежелательных изменений при помощи распределённого доступа; выгрузка и сортировка данных по заданным фильтрам при помощи SQL-запросов; поддержка целостности баз данных, резервное копирование и восстановление после сбоев [15].

Структура базы данных включает следующие основные компоненты: таблицы, в которых есть строки-записи и столбцы-поля, поля с различными типами данных и характеристиками, такими как уникальность или обязательность, связи между таблицами – один-к-одному, один-ко-многим, многие-ко-многим, и индексы для ускорения поиска и сортировки.

Базы данных можно классифицировать по различным признакам. Наиболее важной с точки зрения проектирования является классификация по модели данных – то есть по способу организации и связывания данных между собой [15].

1. В иерархических базах данные организованы в виде дерева, где каждый элемент имеет одного родителя и может иметь несколько потомков. Классическим примером является файловая система компьютера. Иерархические БД были популярны в ранних вычислительных системах, но их главный недостаток – сложность реализации связей «многие-ко-многим».

2. Сетевые модели являются развитием иерархических и позволяют каждому элементу иметь несколько родителей. Это делает их более гибкими, но значительно усложняет структуру и навигацию по данным. В настоящее время сетевые БД используются редко.

3. Реляционные базы данных являются доминирующим типом в современных информационных системах. Данные в них организованы в виде двумерных таблиц, где строки представляют отдельные записи,

а столбцы – атрибуты этих записей. Связи между таблицами устанавливаются с помощью ключей (первичных и внешних). Реляционные БД обладают строгим математическим обоснованием и поддерживают язык запросов SQL.

4. Объектные базы данных ориентированы на хранение объектов в том виде, в котором они используются в объектно-ориентированных языках программирования. Они позволяют хранить не только данные, но и методы их обработки. Объектно-реляционные базы данных сочетают свойства реляционных и объектных моделей [3].

Как видно из приведённой классификации, в настоящее время существует несколько моделей данных, каждая из которых имеет свои преимущества и области применения. Однако многолетняя практика проектирования информационных систем показывает, что для большинства приложений, включая веб-сервисы электронной коммерции, системы учёта и документооборота, наиболее подходящей является реляционная модель данных. Реляционные базы данных обеспечивают целостность, поддержку транзакций и гибкость при выполнении сложных запросов. Поэтому теперь детально рассмотрим основные принципы реляционных баз данных и процесс их проектирования.

1.5 Реляционные базы данных и их проектирование

Реляционная модель данных была предложена Эдгаром Коддом в 1969–1970 годах и до настоящего времени остаётся доминирующей моделью для организации данных в информационных системах. Термин «реляционная» происходит от английского слова relation (отношение), которым в математике обозначают таблицу. Именно табличное представление данных является основой реляционной модели [15].

В реляционной базе данных вся информация представляется в виде двумерных таблиц, которые называются отношениями. Каждая таблица состоит из строк (кортежей) и столбцов (атрибутов). Строка таблицы соответствует одному экземпляру сущности. Столбец таблицы соответствует определённому атрибуту этой сущности.

Ключевым понятием реляционной модели является первичный

ключ – атрибут или набор атрибутов, который однозначно идентифицирует каждую строку в таблице. Значения первичного ключа должны быть уникальными для каждой строки и не могут быть пустыми. Для связывания таблиц между собой используются внешние ключи. Внешний ключ – это атрибут в одной таблице, который ссылается на первичный ключ другой таблицы.

Реляционная модель обладает строгим математическим обоснованием, базирующимся на реляционной алгебре и реляционном исчислении. Практической реализацией этих идей стал язык SQL (Structured Query Language), который является стандартным языком запросов для реляционных СУБД.

Проектирование реляционной базы данных традиционно проходит три последовательных этапа: концептуальное, логическое и физическое проектирование [10].

Концептуальное проектирование – это создание модели предметной области без привязки к конкретной СУБД. На этом этапе выделяются сущности, их атрибуты и связи между сущностями. Результатом концептуального проектирования является ER-диаграмма.

Логическое проектирование – это преобразование концептуальной модели в реляционную схему. Сущности превращаются в таблицы, атрибуты – в поля, связи – во внешние ключи. Также на этом этапе проводится нормализация схемы для устранения избыточности и аномалий.

Физическое проектирование – это реализация логической схемы в конкретной СУБД. Выбираются типы данных, создаются индексы, определяются ограничения целостности, триггеры и хранимые процедуры. Результатом являются SQL-скрипты для создания базы данных [7].

Нормализация – это процесс приведения схемы базы данных к виду, обеспечивающему минимальную избыточность и отсутствие аномалий при вставке, обновлении и удалении данных. Процесс нормализации выполняется поэтапно, и каждый этап соответствует определённой нормальной форме [5].

Первая нормальная форма (1НФ) требует, чтобы все атрибуты таблицы были атомарными (неделимыми), а в таблице не было повторяющихся групп.

Вторая нормальная форма (2НФ) применяется только к таблицам с составным первичным ключом. Она требует, чтобы все неключевые атрибуты зависели от всего составного ключа, а не от его части.

Третья нормальная форма (3НФ) требует, чтобы неключевые атрибуты зависели только от первичного ключа и не зависели от других неключевых атрибутов. Иными словами, в таблице не должно быть транзитивных зависимостей. На практике схему базы данных обычно приводят как минимум к третьей нормальной форме.

Для обеспечения корректности и непротиворечивости данных в реляционных базах данных используются различные ограничения целостности: ограничение уникальности – UNIQUE – гарантирует, что значения в указанном поле не повторяются в разных строках; Ограничение – NOT NULL – требует, чтобы поле обязательно было заполнено; Ограничение – CHECK – позволяет задать произвольное условие, которому должны удовлетворять значения в поле.

Индекс – это структура данных, которая ускоряет выполнение операций поиска и сортировки в таблице. По своей сути индекс напоминает алфавитный указатель в книге [7].

Таким образом, мы рассмотрели основные принципы реляционных баз данных: табличную структуру, ключи, этапы проектирования, нормализацию, ограничения целостности и индексы. Эти теоретические положения являются фундаментом для создания любой реляционной базы данных.

1.6 Выбор СУБД

После рассмотрения теоретических основ реляционных баз данных возникает практический вопрос: какую конкретную систему управления базами данных выбрать для реализации проекта. Для веб-сервиса обмена книгами «BookSwap» в качестве основной СУБД выбрана PostgreSQL.

PostgreSQL позиционируется как самая продвинутая реляционная СУБД с открытым исходным кодом. Она разрабатывается с 1996 года и к настоящему времени приобрела репутацию надёжной, функционально

насыщенной и масштабируемой системы. PostgreSQL поддерживает расширенные возможности, такие как пользовательские типы данных, наследование таблиц, оконные функции и общие табличные выражения [14].

Ключевым преимуществом PostgreSQL для проекта «BookSwap» является поддержка геопространственных данных через расширение PostGIS. PostGIS добавляет в PostgreSQL специальные типы данных для работы с географическими объектами, а также функции для вычисления расстояний и поиска ближайших объектов.

PostgreSQL также предоставляет мощные средства полнотекстового поиска, включая поддержку GIN-индексов и словарей стоп-слов. Это позволяет реализовать быстрый поиск книг по названию, автору и ключевым словам без подключения дополнительных поисковых движков [14].

В области транзакций PostgreSQL полностью соответствует требованиям ACID (атомарность, согласованность, изоляция, долговечность) даже при высоком уровне конкурентного доступа. СУБД поддерживает различные уровни изоляции транзакций и механизмы блокировок, что важно для корректной обработки запросов на обмен – ситуаций, когда два пользователя одновременно пытаются запросить одну и ту же книгу.

PostgreSQL поддерживает создание хранимых процедур и функций на нескольких языках. Это позволяет реализовать сложную бизнес-логику непосредственно на уровне базы данных.

Таким образом, на основании рассмотренных преимуществ для реализации веб-сервиса обмена книгами «BookSwap» выбрана СУБД PostgreSQL с расширением PostGIS. Этот выбор обеспечивает поддержку геопространственных запросов для поиска книг поблизости, мощный полнотекстовый поиск по каталогу, надёжные транзакции для корректной обработки запросов на обмен, а также гибкость и масштабируемость для дальнейшего развития проекта.

Глава 2. Практическая часть

2.1 Описание предметной области

Обмен книгами между людьми существует давно: стихийные группы в социальных сетях, доски объявлений, буккроссинг-полки в кафе и библиотеках. Проблема у всех этих форматов одна: нет никакой инфраструктуры. Человек не знает, кто перед ним, нет истории сделок, нет способа найти конкретную книгу рядом с домом, нет гарантий что обмен вообще состоится.

Сервис BookSwar проектируется чтобы закрыть именно эти проблемы. Пользователи размещают книги, которые готовы отдать в обмен, указывают геолокацию экземпляра, находят подходящие предложения поблизости и договариваются об обмене внутри платформы. Ключевое отличие от существующих аналогов: встроенный геопоиск, система рейтингов на основе реальных завершённых обменов и вишлист с автоматическими уведомлениями, когда нужная книга появляется рядом.

Среди существующих сервисов ближе всего подходят следующие аналоги.

Авито: крупнейшая площадка объявлений, где книги можно отдать бесплатно или продать. Геопоиска в привязке к книгам нет, рейтинг пользователя никак не связан с фактом завершённого обмена, вишлист с уведомлениями отсутствует полностью.

Буккроссинг (bookcrossing.com): сервис для безвозмездной передачи книг через физические точки. Принцип обмена книга-на-книгу не поддерживается, договориться о встрече через платформу невозможно, история взаимодействий не ведётся.

PaperBackSwar: американская платформа для обмена книгами по почте. Наиболее близка по концепции, но не имеет геолокационного поиска, рассчитана на пересылку, а не на личную встречу, и ориентирована исключительно на англоязычный рынок.

Ни один из существующих сервисов не объединяет в одном месте геопоиск, рейтинги участников, автоматизированный учёт обменов и

вишлист с уведомлениями. Именно этот набор функций определяет структуру базы данных BookSwap.

Для понимания логики взаимодействия сущностей рассмотрим три основных пользовательских сценария.

Сценарий 1: добавление книги. Пользователь регистрируется, подтверждает почту и получает статус верифицированного. После этого он добавляет книгу в каталог: указывает название, автора, жанр и геолокацию, то есть, где физически находится экземпляр. Книга получает статус `available` и становится видима другим пользователям в радиусе поиска.

Сценарий 2: запрос на обмен. Пользователь А ищет книгу в радиусе 5 км от своего местоположения, находит нужный экземпляр у пользователя Б. Он создаёт запрос на обмен, предлагая свою книгу взамен. Оба экземпляра получают статус `in_exchange`: пока идут переговоры, их нельзя предложить другим. Пользователь Б принимает или отклоняет запрос. При принятии фиксируется завершённый обмен.

Сценарий 3: вишлист и уведомление. Пользователь добавляет книгу в вишлист, то есть он хочет её найти, но подходящего предложения пока нет. Когда другой пользователь добавляет этот экземпляр в радиусе досягаемости, система автоматически создаёт уведомление и отправляет его первому пользователю.

Все пользователи системы равноправны: отдельной роли администратора или модератора в базе данных не предусмотрено. При этом у пользователя есть два состояния. Первое: сразу после регистрации аккаунт создан, но почта ещё не подтверждена, флаг `is_verified` равен `false`. В этом состоянии пользователь не может инициировать запросы на обмен. Второе: верифицированный пользователь, почта подтверждена, `is_verified` становится `true`, фиксируется время верификации. Токены подтверждения почты и сессионные JWT-токены хранятся вне базы данных в Redis, что снижает нагрузку на PostgreSQL и упрощает управление сроком их жизни.

Исходя из описанной логики, в базе данных нужно хранить авторов и книги как справочник, экземпляры книг у конкретных пользователей с геолокацией, запросы на обмен и факты завершённых обменов, отзы-

вы, вишлисты и уведомления.

2.2 ER-диаграмма базы данных

База данных BookSwap состоит из девяти таблиц, которые можно разбить на четыре смысловые группы: справочник книг, пользователи и их экземпляры, блок обмена и социальный слой. ER-диаграмма представлена на рисунке 2.

Справочник формируют две таблицы. В `authors` хранится информация об авторе: полное имя и краткая биография. Таблица `books` содержит название, жанр, ISBN и год издания, а также внешний ключ `author_id`. Один автор может быть привязан к нескольким книгам, одна книга имеет ровно одного автора.

Таблица `users` описывает зарегистрированных участников сервиса. Помимо стандартных полей аутентификации хранятся два флага состояния: `is_active` показывает не заблокирован ли аккаунт, `is_verified` фиксирует факт подтверждения почты. Поле `rating` содержит средний балл пользователя, который пересчитывается на основе записей в таблице `reviews` после каждого завершённого обмена.

Экземпляры книг у конкретных пользователей хранятся в `user_books`. Каждая запись связывает пользователя и книгу из справочника, содержит поле `location` типа `GEOGRAPHY(POINT)` для хранения геолокации физического местонахождения экземпляра, и поле `status` с тремя возможными значениями: `available` означает что книга доступна для обмена, `in_exchange` означает что по ней уже есть активный запрос, `given_away` фиксирует завершённую передачу.

Блок обмена состоит из двух таблиц. `Exchange_requests` хранит сам запрос: кто инициатор, какую книгу предлагает (`offered_book_id`), какую хочет получить (`requested_book_id`), и текущий статус. Разделение на две таблицы принципиально: `exchange_requests` фиксирует весь жизненный цикл переговоров включая отклонённые запросы, тогда как `exchanges` содержит только реально завершённые обмены.

Социальный слой включает три таблицы. `Reviews` хранит отзывы после завершённого обмена: оба участника могут оставить оценку от 1

до 5 и текстовый комментарий. Таблица wishlists фиксирует желаемые книги пользователя с ограничением уникальности по паре (user_id, book_id). Notifications хранит уведомления с полем type для разграничения причин: совпадение по вишлисту, принятие запроса на обмен и другие события.

Схема нормализована до третьей нормальной формы: все неключевые атрибуты зависят исключительно от первичного ключа своей таблицы, транзитивных зависимостей нет.

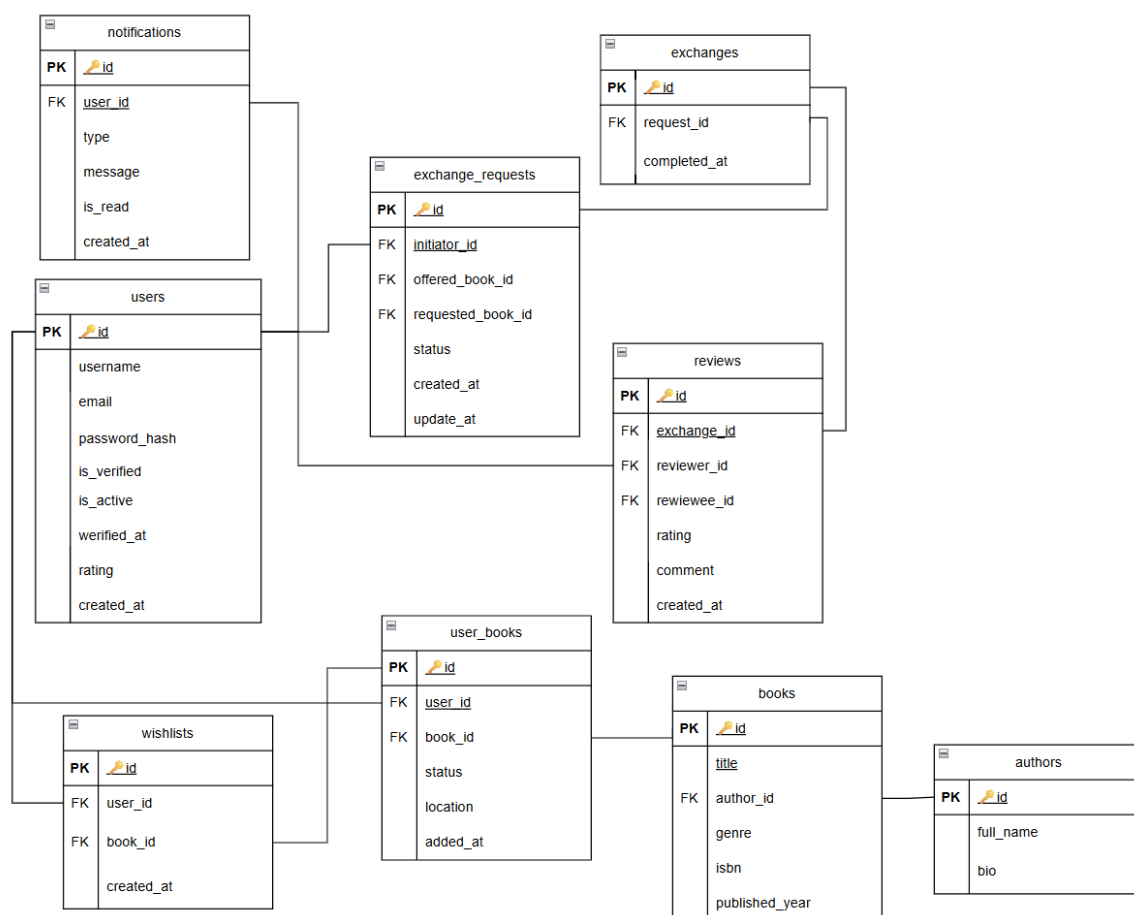


Рисунок 4 - ER-диаграмма базы данных BookSwap

2.3 Структура таблиц и ограничения целостности

На основе разработанной ER-диаграммы и логической модели данных была спроектирована физическая структура базы данных, включающая набор взаимосвязанных таблиц. Каждая таблица соответствует определённой сущности предметной области и содержит атрибуты, описывающие её свойства. Для обеспечения корректности, непротиво-

речивости и надёжности хранимых данных в структуру таблиц интегрированы ограничения целостности (первичные и внешние ключи, ограничения уникальности, проверки допустимых значений, а также запрет на пустые значения).

Ниже приведено детальное описание каждой таблицы с указанием её назначения, перечня полей, типов данных и наложенных ограничений.

Таблица 1 - authors

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор автора
full_name	VARCHAR(255)	NOT NULL	Полное имя автора
bio	TEXT		Краткая биография, необязательное поле

Данная таблица представляет собой справочник авторов, зарегистрированных в системе. Её структура намеренно минималистична и включает всего два информационных поля: `full_name` (полное имя автора) и опциональное поле `bio` (краткая биографическая справка). Такое решение обусловлено особенностями предметной области – сервис ориентирован на обмен физическими книгами, а не на создание энциклопедического каталога авторов. Основные операции (поиск по автору, фильтрация каталога) не требуют детальной биографии. Расширение таблицы дополнительными полями привело бы к неоправданному усложнению схемы без практической пользы для пользователей.

Первичным ключом таблицы является поле `id` (тип `SERIAL`). Поле `full_name` имеет ограничение `NOT NULL`, так как имя автора – обязательный атрибут. Поле `bio` такого ограничения не имеет.

Таблица 2 - books

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор книги
title	VARCHAR(500)	NOT NULL	Название книги
author_id	INTEGER	FK → authors, NOT NULL	Ссылка на автора
genre	VARCHAR(100)		Жанр книги, необязательное поле
isbn	VARCHAR(20)	UNIQUE	Международный номер книги
published_year	SMALLINT		Год издания

Таблица books представляет собой справочник книг, зарегистрированных пользователями в системе. Поле isbn объявлено с ограничением UNIQUE, что гарантирует отсутствие дублирующихся записей об одном и том же издании. Поле author_id является внешним ключом, связывающим книгу с единственным автором из справочника authors. Если у книги несколько авторов, по условиям проекта указывается основной (первый) – этого достаточно для идентификации издания в контексте поиска и обмена.

Первичным ключом таблицы является поле id (тип SERIAL). Поля title и author_id имеют ограничение NOT NULL, так как название книги и автор являются обязательными атрибутами. Поля genre, isbn и published_year такого ограничения не имеют.

Таблица 3 - users

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор пользователя
username	VARCHAR(100)	UNIQUE, NOT NULL	Имя пользователя в системе
email	VARCHAR(255)	UNIQUE, NOT NULL	Электронная почта
password_hash	VARCHAR(255)	NOT NULL	Хэш пароля
is_verified	BOOLEAN	DEFAULT false	Подтверждена ли почта
is_active	BOOLEAN	DEFAULT true	Активен ли аккаунт
verified_at	TIMESTAMPTZ		Дата и время верификации почты
rating	NUMERIC(3,2)	DEFAULT 0	Средний рейтинг пользователя
created_at	TIMESTAMPTZ	DEFAULT now()	Дата регистрации

Таблица users предназначена для хранения учётных записей зарегистрированных пользователей сервиса. Поля email и username объявлены с ограничением UNIQUE, что гарантирует уникальность адреса электронной почты и логина в системе. Пароль (password_hash) хранится исключительно в виде хэша – прямой текст пароля не сохраняется в целях безопасности.

Поля is_verified и is_active представляют собой булевы флаги, управляющие доступом пользователя к функциям системы. Неверифицированный пользователь (флаг is_verified равен false) не может создавать запросы на обмен, что стимулирует подтверждение контактных данных. Заблокированный пользователь (флаг is_active равен false) не может выполнять никаких действий в системе. Поле rating хранит средний рейтинг пользователя, вычисляемый на основе полученных отзывов. Первичным ключом таблицы является поле id (тип SERIAL). По-

ля username, email, password_hash и created_at имеют ограничение NOT NULL.

Таблица 4 - user_books

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор экземпляра
user_id	INTEGER	FK → users, NOT NULL	Владелец экземпляра
book_id	INTEGER	FK → books, NOT NULL	Ссылка на книгу в справочнике
status	book_status	NOT NULL, CHECK	available / in_exchange / given_away
location	GEOGRAPHY(POINT)	NOT NULL	Геолокация экземпляра (PostGIS)
added_at	TIMESTAMPTZ	DEFAULT now()	Дата добавления

Таблица user_books является центральной для сервиса, поскольку именно она связывает конкретного пользователя с конкретной книгой и добавляет к этой паре ключевые атрибуты – геолокацию и статус. Поле location хранит географические координаты в формате GEOGRAPHY(POINT) расширения PostGIS, что позволяет выполнять пространственные запросы для поиска книг в заданном радиусе от текущего местоположения пользователя. Поля user_id и book_id являются внешними ключами, ссылающимися на таблицы users и books соответственно.

Поле status представляет собой перечисление (book_status) с тремя возможными значениями, контролируемое ограничением CHECK. Статус available означает, что книга доступна для обмена и отображается в результатах поиска. Статус in_exchange временно блокирует книгу от

новых запросов на время, пока она участвует в активном обмене. Статус `given_away` фиксирует факт завершённой передачи книги. Первичным ключом таблицы является поле `id` (тип `SERIAL`). Поля `user_id`, `book_id`, `status` и `location` имеют ограничение `NOT NULL`.

Таблица 5 - `exchange_requests`

Поле	Тип данных	Ограничения	Описание
id	<code>SERIAL</code>	<code>PRIMARY KEY</code>	Уникальный идентификатор запроса
initiator_id	<code>INTEGER</code>	<code>FK → users,</code> <code>NOT NULL</code>	Пользователь, создавший запрос
offered_book_id	<code>INTEGER</code>	<code>FK →</code> <code>user_books,</code> <code>NOT NULL</code>	Предлагаемая книга инициатора
requested_book_id	<code>INTEGER</code>	<code>FK →</code> <code>user_books,</code> <code>NOT NULL</code>	Желаемая книга у второй стороны
status	<code>request_status</code>	<code>NOT NULL,</code> <code>CHECK</code>	<code>pending / accepted / declined / cancelled</code>
created_at	<code>TIMESTAMPTZ</code>	<code>DEFAULT now()</code>	Дата создания запроса
updated_at	<code>TIMESTAMPTZ</code>		Дата последнего изменения статуса

Таблица `exchange_requests` хранит все запросы на обмен, независимо от их исхода (активные, завершённые, отклонённые или отменённые). Два внешних ключа – `offered_book_id` и `requested_book_id` – ссылаются на разные записи в таблице `user_books`. Это означает, что одна запись в `exchange_requests` связывает книгу, которую пользователь предлагает отдать, с книгой, которую он хочет получить взамен. Каждый из этих двух экземпляров принадлежит разным пользователям. Поле `initiator_id` указывает на пользователя, который создал запрос.

Поле `status` представляет собой перечисление (`request_status`) со

значениями pending (ожидает подтверждения), accepted (подтверждён), declined (отклонён) и cancelled (отменён), контролируемое ограничением CHECK. Поле updated_at не заполняется вручную – его значение автоматически обновляется триггером при каждом изменении статуса запроса, что позволяет отслеживать динамику обработки заявки. Первичным ключом таблицы является поле id (тип SERIAL). Поля initiator_id, offered_book_id, requested_book_id, status и created_at имеют ограничение NOT NULL.

Таблица 6 - exchanges

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор обмена
request_id	INTEGER	FK, UNIQUE	Исходный запрос на обмен
completed_at	TIMESTAMPTZ	DEFAULT now()	Дата завершения обмена

Таблица exchanges является таблицей фактов – сюда попадают только те запросы на обмен, которые успешно завершились передачей книг. Поле request_id является внешним ключом, ссылающимся на таблицу exchange_requests, и имеет ограничение UNIQUE. Это исключает ситуацию, когда один и тот же запрос порождает два разных завершённых обмена, гарантируя, что каждая заявка может быть успешно закрыта только один раз.

Поле completed_at фиксирует дату и время завершения обмена. На таблицу exchanges ссылаются отзывы: отзыв можно написать только по завершённому обмену. Первичным ключом таблицы является поле id. Поля request_id и completed_at имеют ограничение NOT NULL.

Таблица 7 - reviews

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор отзыва
exchange_id	INTEGER	FK, NOT NULL	Обмен, по итогам которого написан отзыв
reviewer_id	INTEGER	FK, NOT NULL	Автор отзыва
reviewee_id	INTEGER	FK, NOT NULL	Пользователь, о котором отзыв
rating	SMALLINT	NOT NULL, CHECK (1-5)	Оценка от 1 до 5
comment	TEXT		Текст отзыва
created_at	TIMESTAMPTZ	DEFAULT now()	Дата публикации отзыва

Таблица reviews предназначена для хранения отзывов, которые участники обмена оставляют друг о друге. Поля reviewer_id и reviewee_id являются внешними ключами, ссылающимися на таблицу users – первый указывает на того, кто пишет отзыв, второй на того, о ком пишут. Каждый участник завершённого обмена может оставить независимый отзыв о другом, что позволяет формировать двустороннюю репутацию. Поле exchange_id является внешним ключом, связывающим отзыв с конкретным завершённым обменом в таблице exchanges.

Поле rating имеет ограничение CHECK (rating >= 1 AND rating <= 5), которое проверяется непосредственно при вставке записи, гарантируя, что оценка всегда находится в диапазоне от 1 до 5 звёзд. Поле comment не является обязательным и может содержать произвольный текст. Первичным ключом таблицы является поле id (тип SERIAL). Поля exchange_id, reviewer_id, reviewee_id и rating имеют ограничение NOT NULL. Рейтинг пользователя вычисляется как среднее арифметическое всех оценок, полученных в поле reviewee_id.

Таблица 8 - wishlists

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор записи
user_id	INTEGER	FK, NOT NULL	Пользователь, добавивший книгу в вишлист
book_id	INTEGER	FK , NOT NULL	Желаемая книга
created_at	TIMESTAMPTZ	DEFAULT now()	Дата добавления в вишлист

Таблица wishlists позволяет пользователям отслеживать книги, которые они хотели бы получить в будущем. Поля user_id и book_id являются внешними ключами, ссылающимися на таблицы users и books соответственно. Составное поле (user_id, book_id) объявлено с ограничением UNIQUE – это гарантирует, что одна и та же книга не может быть добавлена в вишлист одного пользователя дважды.

При добавлении нового экземпляра книги в таблицу user_books приложение автоматически проверяет, есть ли эта книга в чьих-либо вишлистах. При обнаружении совпадения по book_id создаётся уведомление для заинтересованного пользователя о том, что желаемая книга появилась у другого участника поблизости. Первичным ключом таблицы является поле id (тип SERIAL). Поля user_id и book_id имеют ограничение NOT NULL.

Таблица 9 - notifications

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор уведомления
user_id	INTEGER	FK, NOT NULL	Получатель уведомления
type	VARCHAR(50)	NOT NULL	Тип уведомления (wishlist_match и др.)
message	TEXT	NOT NULL	Текст уведомления
is_read	BOOLEAN	DEFAULT false	Прочитано ли уведомление
created_at	TIMESTAMPTZ	DEFAULT now()	Дата создания уведомления

Таблица notifications предназначена для хранения всех уведомлений, отправленных пользователям системы. Уведомления не удаляются после прочтения – вместо этого флаг is_read переключается из false в true, а сама запись остаётся в базе данных, сохраняя полную историю взаимодействия пользователя с системой. Поле user_id является внешним ключом, ссылающимся на таблицу users.

Поле type (тип VARCHAR(50)) разграничивает причины отправки уведомлений: wishlist_match, exchange_accepted и другие события. Это позволяет на стороне приложения фильтровать и группировать уведомления по типу без необходимости разбора текстового сообщения. Первичным ключом таблицы является поле id (тип SERIAL). Поля user_id, type и message имеют ограничение NOT NULL.

Описанная логическая структура таблиц и ограничений целостности требует физической реализации в выбранной СУБД. Для этого используются DDL-скрипты (Data Definition Language), которые создают таблицы, типы данных и вспомогательные объекты базы данных. В следующем разделе рассматривается логика построения этих скриптов и обосновывается порядок создания объектов.

2.4 DDL-скрипты создания базы данных

DDL (Data Definition Language) представляет собой подмножество языка SQL, предназначенное для определения структуры базы данных – создания таблиц, типов данных, ограничений целостности и вспомогательных объектов. Полные скрипты реализации приведены в Приложении А, в данном разделе рассматривается логика построения каждого блока.

Перед созданием таблиц подключаются два расширения PostgreSQL. Расширение PostGIS добавляет поддержку географических типов и пространственных функций – без него невозможно хранить координаты в поле `location` и выполнять запросы на вычисление расстояния. Расширение `pg_trgm` обеспечивает поддержку триграммных индексов для полнотекстового поиска книг по названию и автору

Листинг 1 – Подключение расширений

```
CREATE EXTENSION postgis;  
CREATE EXTENSION pg_trgm;
```

Для полей `status` используются пользовательские ENUM-типы. В отличие от VARCHAR с CHECK-ограничением, ENUM явно перечисляет допустимые значения на уровне типа данных. PostgreSQL отклонит любое значение, не входящее в список, ещё до выполнения операции вставки.

Листинг 2 – Создание пользовательских типов статусов

```
CREATE TYPE book_status AS ENUM  
(  
    'available',  
    'in_exchange',  
    'given_away');  
  
CREATE TYPE request_status AS ENUM  
(  
    'pending',  
    'accepted',  
    'declined',  
    'cancelled');
```

Таблицы создаются в порядке зависимостей по внешним ключам: сначала те, на которые ссылаются другие. `authors` и `books` создаются в

числе первых, users – раньше user_books и всех таблиц, где участвуют пользователи. exchanges создаётся после exchange_requests, reviews – после exchanges.

Листинг 3 – Порядок создания таблиц

```
CREATE TABLE authors;  
CREATE TABLE books; - ссылается на authors  
CREATE TABLE users;  
CREATE TABLE user_books; - ссылается на users, books  
CREATE TABLE exchange_requests; - ссылается на users, user_books  
CREATE TABLE exchanges; - ссылается на exchange_requests  
CREATE TABLE reviews; - ссылается на exchanges, users  
CREATE TABLE wishlists; - ссылается на users, books  
CREATE TABLE notifications; - ссылается на users
```

Для поля updated_at в таблице exchange_requests создаётся триггер, который автоматически проставляет текущее время при каждом обновлении записи. Это освобождает приложение от необходимости передавать данное значение явно при каждом изменении статуса запроса.

Листинг 4 – Создание функции и триггера для автообновления updated_at

```
CREATE OR REPLACE FUNCTION set_updated_at()  
RETURNS TRIGGER AS $$  
BEGIN  
NEW.updated_at = NOW();  
RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
CREATE TRIGGER update_exchange_requests_updated_at  
BEFORE UPDATE ON exchange_requests  
FOR EACH ROW  
EXECUTE FUNCTION set_updated_at();
```

При необходимости пересоздать схему таблицы удаляются в порядке, обратном созданию. Параметр CASCADE автоматически удаляет зависимые объекты.

Листинг 5 – Порядок удаления таблиц

```
DROP TABLE IF EXISTS notifications, wishlists, reviews;  
DROP TABLE IF EXISTS exchanges, exchange_requests;  
DROP TABLE IF EXISTS user_books, users;  
DROP TABLE IF EXISTS books, authors;  
DROP TYPE IF EXISTS request_status, book_status;
```

Спроектированная структура таблиц и ограничения целостности обеспечивают корректное хранение данных. Однако для приемлемой производительности при росте объёма информации этого недостаточно. Особенно чувствительными к скорости выполнения являются операции геопоиска, полнотекстового поиска и соединения таблиц по внешним ключам. Для решения этих задач в PostgreSQL используются различные типы индексов, выбор которых обосновывается в следующем разделе.

2.5 Индексы

Индекс – это вспомогательная структура данных, которая позволяет PostgreSQL находить нужные строки без последовательного перебора всей таблицы. В BookSwar три типа запросов особенно чувствительны к производительности: поиск книг в заданном радиусе, поиск по названию или автору и фильтрация экземпляров по статусу. Под каждый подбирается свой тип индекса. Полные DDL-скрипты вынесены в Приложение Б.

GiST-индекс на поле `location` таблицы `user_books` является главным индексом проекта. Стандартный B-tree не умеет работать с географическими данными, потому что не понимает понятие расстояния между координатами. GiST строит пространственное дерево, которое разбивает карту на иерархические регионы и позволяет функции `ST_DWithin` мгновенно отсеять все точки за пределами заданного радиуса.

Листинг 6 – Пространственный индекс

```
CREATE INDEX idx_user_books_location ON user_books USING GIST  
(location);
```

Для полнотекстового поиска книг по названию и авторов по имени используются GIN-индексы с расширением `pg_trgm`. Триграммный подход разбивает строку на трёхсимвольные фрагменты и индексирует каждый из них, что позволяет эффективно выполнять запросы с `LIKE '%подстрока%'`. Обычный B-tree в таких случаях бесполезен: он работает только с префиксными совпадениями.

Листинг 7 – Триграммные индексы для текстового поиска

```
CREATE INDEX idx_books_title ON books USING GIN (title gin_trgm_ops);
CREATE INDEX idx_authors_full_name ON authors USING GIN (full_name
gin_trgm_ops);
```

PostgreSQL автоматически создаёт B-tree индексы только для первичных ключей. Внешние ключи индексами не покрываются, хотя именно по ним чаще всего выполняются JOIN-операции. Для каждого внешнего ключа создаётся отдельный B-tree индекс.

Листинг 8 – B-tree индексы на внешние ключи

```
CREATE INDEX idx_books_author_id ON books(author_id);
CREATE INDEX idx_user_books_user_book ON user_books(user_id, book_id);
CREATE INDEX idx_exchange_requests_initiator ON exchange_requests(
    initiator_id);
CREATE INDEX idx_exchange_requests_offered ON exchange_requests(
    offered_book_id);
CREATE INDEX idx_exchange_requests_requested ON exchange_requests(
    requested_book_id);
CREATE INDEX idx_exchanges_request_id ON exchanges(request_id);
CREATE INDEX idx_reviews_exchange_id ON reviews(exchange_id);
CREATE INDEX idx_reviews_reviewer_id ON reviews(reviewer_id);
CREATE INDEX idx_reviews_reviewee_id ON reviews(reviewee_id);
CREATE INDEX idx_wishlists_user_id ON wishlists(user_id);
CREATE INDEX idx_wishlists_book_id ON wishlists(book_id);
CREATE INDEX idx_notifications_user_id ON notifications(user_id);
```

Частичный индекс на поле `status` таблицы `user_books` индексирует только записи со статусом `available`. Записи со статусами `in_exchange` и `given_away` в поиске не участвуют, поэтому включать их в индекс нет смысла. Частичный индекс меньше по размеру и быстрее обновляется.

Листинг 9 – Частичный индекс (только доступные экземпляры)

```
CREATE INDEX idx_user_books_available ON user_books(user_id) WHERE  
status = 'available';
```

Сводная таблица всех созданных индексов представлена в таблице 10 (Приложение 2).

Разработанные индексы призваны обеспечить высокую производительность основных операций сервиса. Для проверки их эффективности и демонстрации функциональности спроектированной схемы данных ниже приведены примеры ключевых запросов, каждый из которых иллюстрирует один из сценариев работы платформы BookSwap.

2.6 Примеры запросов

Для проверки работоспособности спроектированной схемы данных и оценки эффективности созданных индексов необходимо выполнить набор запросов, имитирующих реальные сценарии использования сервиса. В данном разделе приведены пять ключевых запросов, охватывающих основные функции платформы BookSwap: геопоиск доступных книг в заданном радиусе, полнотекстовый поиск по каталогу, получение списка активных запросов на обмен, формирование вишлиста с подсчётом доступных экземпляров и вычисление рейтинга пользователя. Для каждого запроса представлена упрощённая логическая форма с пояснением принципа работы. Полные рабочие тексты запросов вынесены в Приложение 3.

Листинг 10 – Поиск книг в заданном радиусе

```
SELECT title, author, owner, owner_rating, distance_km  
FROM user_books  
JOIN books, authors, users  
WHERE status = 'available' AND  
ST_DWithin(location, user_location, 5000)  
ORDER BY distance_km;
```

Данный запрос является основным для сервиса. Он выбирает из таблицы `user_books` все экземпляры книг со статусом `available`, расстояние

до которых от переданных координат пользователя не превышает 5 км. Посредством соединения с таблицами books, authors и users возвращается полная информация о книге и её владельце. Запрос опирается на GiST-индекс поля location: функция ST_DWithin использует пространственное дерево, что исключает необходимость полного перебора всех строк таблицы.

Листинг 11 – Полнотекстовый поиск по названию и автору

```
SELECT title, author, genre, published_year, relevance
FROM books
JOIN authors
WHERE title ILIKE '%query%' OR author ILIKE '%query%'
ORDER BY similarity(title || ' ' || author, query) DESC LIMIT 20;
```

Запрос осуществляет поиск книг по подстроке одновременно в поле title таблицы books и поле full_name таблицы authors. Оператор ILIKE обеспечивает поиск без учёта регистра символов. Для ранжирования результатов используется функция similarity из расширения pg_trgm, возвращающая значение от 0 до 1, где 1 соответствует точному совпадению. GIN-индексы, построенные на обоих текстовых полях, обеспечивают высокую производительность запроса без полного сканирования таблиц.

Листинг 12 – Активные запросы на обмен пользователя

```
SELECT request_id, status, initiator, offered_book, requested_book,
CASE WHEN initiator = current_user THEN
'outgoing' ELSE
'incoming' END AS direction
FROM exchange_requests
JOIN user_books AS offered, user_books AS requested, books, users
WHERE status = 'pending' AND (initiator = current_user OR requested_book_owner
= current_user)
ORDER BY created_at DESC;
```

Запрос возвращает все запросы со статусом pending, в которых пользователь участвует либо в качестве инициатора, либо в качестве владельца запрашиваемой книги. Цепочка соединений проходит через таблицу user_books дважды с использованием различных псевдонимов:

offered – для предлагаемой книги, requested – для запрашиваемой. Поле direction вычисляется посредством конструкции CASE и отображает направление запроса (исходящий или входящий).

Листинг 13 – Вишлист пользователя с доступными экземплярами

```
SELECT title, author, created_at, COUNT(available_instances) AS
available_count
FROM wishlists
JOIN books, authors
LEFT JOIN user_books ON user_books.book_id = books.id AND user_books.status =
    'available' AND user_books.user_id != current_user
WHERE wishlists.user_id = current_user
GROUP BY books.id
ORDER BY available_count DESC;
```

Запрос выбирает все книги из вишлиста пользователя и для каждой из них подсчитывает количество доступных экземпляров у других участников. Использование LEFT JOIN с таблицей user_books необходимо для того, чтобы книги, не имеющие доступных экземпляров, также попадали в результат с нулевым значением счётчика. Сортировка по убыванию available_count выводит книги с доступными экземплярами в начало списка.

Листинг 14 – Расчёт рейтинга пользователя

```
SELECT username, stored_rating, ROUND(AVG(rating), 2) AS
calculated_rating, COUNT(reviews) AS reviews_count, COUNT(exchanges)
AS exchanges_count
FROM users
LEFT JOIN reviews ON reviews.reviewee_id = users.id
LEFT JOIN exchanges ON exchanges.user_id = users.id
WHERE users.id = target_user_id
GROUP BY users.id;
```

Запрос вычисляет актуальный рейтинг пользователя на основе всех полученных им отзывов. Функция AVG по полю rating возвращает среднее арифметическое значение, а ROUND округляет его до двух знаков после запятой. Дополнительно подсчитываются общее количество отзывов и завершённых обменов пользователя. Результат выполнения запроса используется для обновления поля rating в таблице users после добавления каждого нового отзыва.

Таким образом, разработанная база данных PostgreSQL с расширением PostGIS полностью обеспечивает функциональные требования сервиса BookSwar: геолокационный поиск книг, полнотекстовый поиск по каталогу, управление запросами на обмен, формирование вишлиста и расчёт рейтинга пользователей. Представленные примеры запросов демонстрируют корректную работу всех ключевых механизмов системы. На этом практическая часть курсовой работы завершается, основные выводы представлены в заключении.

Заключение

Для раскрытия темы во введении курсового проекта была определена цель – проектирование и разработка базы данных для веб-сервиса обмена книгами между пользователями «BookSwar», обеспечивающей поиск, обмен и учёт книг на основе геолокации и системы рейтингов. В ходе работы были исследованы теоретические основы проектирования информационных систем, методологии разработки, нотации моделирования бизнес-процессов, а также принципы организации и проектирования реляционных баз данных. Были выполнены все поставленные задачи, среди них:

1. Рассмотрены теоретические основы проектирования информационных систем: методологии, нотации моделирования, элементы UML и принципы организации баз данных.
2. Изучена классификация баз данных по модели данных и обоснован выбор реляционной модели для разрабатываемого сервиса.
3. Спроектирована логическая структура базы данных: выделены сущности, определены их атрибуты, первичные и внешние ключи, схема приведена к третьей нормальной форме.
4. Реализована база данных в PostgreSQL с расширением PostGIS, включающая создание таблиц, пользовательских ENUM-типов для статусных полей и ограничений целостности.
5. Создана система индексов для оптимизации производительности запросов: GiST-индекс для геопоиска, GIN-индексы для полнотекстового поиска, B-tree индексы для внешних ключей и частичный индекс для фильтрации книг по статусу.
6. Разработаны и протестированы ключевые запросы, охватывающие основные сценарии работы сервиса. Все запросы продемонстрировали корректную работу.

В результате была спроектирована и реализована база данных PostgreSQL с расширением PostGIS для веб-сервиса обмена книгами «BookSwar». Разработанное решение обеспечивает полную поддержку функциональных требований: геолокационный поиск книг в заданном радиусе, полнотекстовый поиск по каталогу, управление запросами на

обмен с отслеживанием статусов, формирование вишлиста и расчёт рейтинга пользователей на основе полученных отзывов. Созданная система индексов гарантирует высокую производительность даже при значительном объёме данных, а ограничения целостности – надёжность и непротиворечивость хранимой информации.

Разработанная база данных может быть использована в качестве основы для полноценного веб-сервиса обмена книгами, а также может быть масштабирована и доработана с учётом дополнительных требований, таких как мобильное приложение или интеграция с внешними книжными каталогами.

Библиографический список

1. Буч Г., Рамбо Д., Якобсон А. Язык UML. Руководство пользователя / Г. Буч, Д. Рамбо, А. Якобсон. – М.: ДМК Пресс, 2020. – 496 с. – Текст: непосредственный.
2. Вендров А.М. Проектирование программного обеспечения: учебник / А.М. Вендров. – М.: Финансы и статистика, 2019. – 560 с. – Текст: непосредственный.
3. Гарсиа-Молина Г. Системы баз данных. Полный курс / Г. Гарсиа-Молина, Дж. Ульман, Дж. Уидом. – М.: Вильямс, 2019. – 1088 с. – Текст: непосредственный.
4. Гвоздева В.А. Основы построения автоматизированных информационных систем: учебное пособие / В.А. Гвоздева, И.Ю. Лаврентьева. – М.: Форум, 2018. – 320 с. – Текст: непосредственный.
5. Грекул В.И. Проектирование информационных систем: учебное пособие / В.И. Грекул, Г.Н. Денищенко, Н.Л. Коровкина. – М.: Интернет-Университет Информационных Технологий, 2020. – 304 с. – Текст: непосредственный.
6. ГОСТ 20886-85. Организация данных в системах обработки данных. Термины и определения. – М.: Издательство стандартов, 1986. – 40 с. – Текст: непосредственный.
7. Дейт К.Дж. Введение в системы баз данных / К.Дж. Дейт. – М.: Вильямс, 2018. – 1328 с. – Текст: непосредственный.
8. Исаев Г.Н. Информационные системы в экономике: учебник / Г.Н. Исаев. – М.: Омега-Л, 2019. – 462 с. – Текст: непосредственный.
9. Калянов Г.Н. Моделирование, анализ, реорганизация и автоматизация бизнес-процессов: учебник / Г.Н. Калянов. – М.: Финансы и статистика, 2019. – 240 с. – Текст: непосредственный.
10. Коннолли Т. Базы данных. Проектирование, реализация и сопровождение: учебник / Т. Коннолли, К. Бегг. – М.: Вильямс, 2020. – 1440 с. – Текст: непосредственный.
11. Кузнецов А.А. Шеринговая экономика в культуре: обмен книгами // Цифровая экономика. – 2022. – № 2 (14). – С. 33–40. – Текст: непо-

средственный.

12. Ларман К. Применение UML и шаблонов проектирования / К. Ларман. – М.: Вильямс, 2019. – 736 с. – Текст: непосредственный.
13. Орлов С.А. Технологии разработки программного обеспечения: учебник / С.А. Орлов, Б.Я. Цилькер. – СПб.: Питер, 2019. – 672 с. – Текст: непосредственный.
14. PostgreSQL: Документация. – URL: <https://postgrespro.ru/docs/postgresql> (дата обращения: 25.05.2026). – Текст: электронный.
15. Практикум | Блог. Найти, сохранить и защитить: как СУБД помогают аналитикам и маркетологам: [Электронный ресурс]. – Режим доступа: <https://practicum.yandex.ru/blog/chto-takoe-subd/> (дата обращения: 18.05.2026). – Текст: электронный.
16. Рыбальченко О.В. Моделирование бизнес-процессов. Нотации IDEF0, DFD, BPMN: учебное пособие / О.В. Рыбальченко. – СПб.: Университет ИТМО, 2018. – 120 с. – Текст: непосредственный.
17. Соммервилл И. Инженерия программного обеспечения: учебник / И. Соммервилл. – М.: Вильямс, 2017. – 816 с. – Текст: непосредственный.
18. Чен П. Модель «сущность-связь» – шаг к унифицированному представлению данных / П. Чен // СУБД. – 1995. – № 3. – С. 32–45. – Текст: непосредственный.
19. Черников Б.В. Информационные системы в экономике: учебное пособие / Б.В. Черников. – М.: ИНФРА-М, 2020. – 380 с. – Текст: непосредственный.
20. Фаулер М. UML. Основы / М. Фаулер. – СПб.: Символ-Плюс, 2018. – 192 с. – Текст: непосредственный.

Приложение

Приложение 1

Листинг 1.1 – Подключение расширений и создание типов

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE TYPE book_status AS ENUM (
    'available',
    'in_exchange',
    'given_away');

CREATE TYPE request_status AS ENUM (
    'pending',
    'accepted',
    'declined',
    'cancelled');
```

Листинг 1.2 – Создание таблиц

```
CREATE TABLE authors (
    id SERIAL PRIMARY KEY,
    full_name VARCHAR(255) NOT NULL,
    bio TEXT);

CREATE TABLE books (
    id SERIAL PRIMARY KEY,
    title VARCHAR(500) NOT NULL,
    author_id INTEGER NOT NULL REFERENCES authors(id),
    genre VARCHAR(100),
    isbn VARCHAR(20) UNIQUE,
    published_year SMALLINT);

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(100) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    is_verified BOOLEAN NOT NULL DEFAULT false,
    is_active BOOLEAN NOT NULL DEFAULT true,
    verified_at TIMESTAMPTZ,
    rating NUMERIC(3,2) NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now());
```

```

CREATE TABLE user_books (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  book_id INTEGER NOT NULL REFERENCES books(id),
  status book_status NOT NULL DEFAULT
  'available',
  location GEOGRAPHY(POINT, 4326) NOT NULL,
  added_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE exchange_requests (
  id SERIAL PRIMARY KEY,
  initiator_id INTEGER NOT NULL REFERENCES users(id),
  offered_book_id INTEGER NOT NULL REFERENCES user_books(id),
  requested_book_id INTEGER NOT NULL REFERENCES user_books(id),
  status request_status NOT NULL DEFAULT
  'pending',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ);

CREATE TABLE exchanges (
  id SERIAL PRIMARY KEY,
  request_id INTEGER NOT NULL UNIQUE REFERENCES exchange_requests(id),
  completed_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE reviews (
  id SERIAL PRIMARY KEY,
  exchange_id INTEGER NOT NULL REFERENCES exchanges(id),
  reviewer_id INTEGER NOT NULL REFERENCES users(id),
  reviewee_id INTEGER NOT NULL REFERENCES users(id),
  rating SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE wishlists (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  book_id INTEGER NOT NULL REFERENCES books(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (user_id, book_id));

CREATE TABLE notifications (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  type VARCHAR(50) NOT NULL,
  message TEXT NOT NULL,
  is_read BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now());

```


Листинг 1.3 – Триггер автообновления updated_at

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
NEW.updated_at = now();
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_exchange_requests_updated_at
BEFORE UPDATE ON exchange_requests
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

Листинг 1.4 – Удаление схемы

```
DROP TABLE IF EXISTS notifications CASCADE;
DROP TABLE IF EXISTS wishlists CASCADE;
DROP TABLE IF EXISTS reviews CASCADE;
DROP TABLE IF EXISTS exchanges CASCADE;
DROP TABLE IF EXISTS exchange_requests CASCADE;
DROP TABLE IF EXISTS user_books CASCADE;
DROP TABLE IF EXISTS users CASCADE;
DROP TABLE IF EXISTS books CASCADE;
DROP TABLE IF EXISTS authors CASCADE;
DROP TYPE IF EXISTS request_status CASCADE;
DROP TYPE IF EXISTS book_status CASCADE;
```

Приложение 2

Листинг 2.1 – Создание всех индексов

```
CREATE INDEX idx_user_books_location ON user_books USING GIST (location);
CREATE INDEX idx_books_title_trgm ON books USING GIN (title gin_trgm_ops);
CREATE INDEX idx_authors_full_name_trgm ON authors USING GIN (full_name
    gin_trgm_ops);
CREATE INDEX idx_books_author_id ON books (author_id);
CREATE INDEX idx_user_books_user_id ON user_books (user_id);
CREATE INDEX idx_user_books_book_id ON user_books (book_id);
CREATE INDEX idx_exch_req_initiator_id ON exchange_requests (initiator_id);
CREATE INDEX idx_exch_req_offered_book ON exchange_requests(offered_book_id);
CREATE INDEX idx_exch_req_requested_book ON exchange_requests (
    requested_book_id);
CREATE INDEX idx_exchanges_request_id ON exchanges (request_id);
CREATE INDEX idx_reviews_exchange_id ON reviews (exchange_id);
```

```
CREATE INDEX idx_reviews_reviewer_id ON reviews (reviewer_id);
CREATE INDEX idx_reviews_reviewee_id ON reviews (reviewee_id);
CREATE INDEX idx_wishlists_user_id ON wishlists (user_id);
CREATE INDEX idx_wishlists_book_id ON wishlists (book_id);
CREATE INDEX idx_notifications_user_id ON notifications (user_id);
CREATE INDEX idx_user_books_available ON user_books (user_id) WHERE status =
    'available';
```

Таблица 10 - Индексы базы данных BookSwap

Таблица.поле	Тип	Метод	Назначение
user_books.location	Простран- ственный	GiST	Геопоиск в радиусе (ST_DWithin)
books.title	Текстовый	GIN	Поиск по подстроке названия
authors.full_name	Текстовый	GIN	Поиск по подстроке имени автора
books.author_id	Внешний ключ	B-tree	Ускорение JOIN с authors
user_books.user_id	Внешний ключ	B-tree	Ускорение JOIN с users
user_books.book_id	Внешний ключ	B-tree	Ускорение JOIN с books
exchange_requests.initiator_id	Внешний ключ	B-tree	Запросы инициатора
exchange_requests.offered_book_id	Внешний ключ	B-tree	Проверка предлагаемой книги
exchange_requests.requested_book_id	Внешний ключ	B-tree	Проверка желаемой книги
exchanges.request_id	Внешний ключ	B-tree	Связь с exchange_requests

Продолжение таблицы 10

Таблица.поле	Тип	Метод	Назначение
reviews.exchange_id	Внешний ключ	B-tree	Отзывы по обмену
reviews.reviewer_id	Внешний ключ	B-tree	Отзывы от пользователя
reviews.reviewee_id	Внешний ключ	B-tree	Отзывы о пользователе
wishlists.user_id	Внешний ключ	B-tree	Вишлист пользователя
wishlists.book_id	Внешний ключ	B-tree	Книги в вишлистах
notifications.user_id	Внешний ключ	B-tree	Уведомления пользователя
user_books.status='available'	Частичный	B-tree	Только доступные экземпляры

Приложение 3

Листинг 3.1 – Поиск книг в радиусе 5 км

```
SELECT ub.id AS user_book_id, b.title AS book_title, a.full_name AS author, b
    .genre, u.username AS owner, u.rating AS owner_rating, ROUND(ST_Distance(
    ub.location,ST_MakePoint(:lng, :lat)::GEOGRAPHY) /1000.0,2) AS
    distance_km
FROM user_books ub
JOIN books b ON b.id = ub.book_id
JOIN authors a ON a.id = b.author_id
JOIN users u ON u.id = ub.user_id
WHERE ub.status = 'available'AND ST_DWithin(ub.location, ST_MakePoint(:lng, :
    lat)::GEOGRAPHY, 5000)
ORDER BY distance_km;
```

Листинг 3.2 – Полнотекстовый поиск по названию и автору

```
SELECT b.id, b.title, a.full_name AS author, b.genre, b.published_year,  
       GREATEST (similarity (b.title, :query), similarity(a.full_name, :query))  
       AS relevance  
FROM books b  
JOIN authors a ON a.id = b.author_id  
WHERE b.title ILIKE '%' || :query || '%' OR a.full_name ILIKE '%' || :query  
      || '%'  
ORDER BY relevance DESC LIMIT 20;
```

Листинг 3.3 – Активные запросы на обмен пользователя

```
SELECT er.id AS request_id, er.status, er.created_at, initiator.username AS  
       initiator, bo.title AS offered_book, br.title AS requested_book,  
CASE  
WHEN er.initiator_id = :user_id THEN 'outgoing'  
ELSE 'incoming'  
END AS direction  
FROM exchange_requests er  
JOIN users initiator ON initiator.id = er.initiator_id  
JOIN user_books offered ON offered.id = er.offered_book_id  
JOIN user_books requested ON requested.id = er.requested_book_id  
JOIN books bo ON bo.id = offered.book_id  
JOIN books br ON br.id = requested.book_id  
WHERE er.status = 'pending' AND (er.initiator_id = :user_id OR requested.  
       user_id = :user_id)  
ORDER BY er.created_at DESC;
```

Листинг 3.4 – Вишлист пользователя с доступными экземплярами

```
SELECT b.id, b.title, a.full_name AS author, b.genre, w.created_at AS  
       added_to_wishlist, COUNT(ub.id) AS available_count  
FROM wishlists w  
JOIN books b ON b.id = w.book_id  
JOIN authors a ON a.id = b.author_id  
LEFT JOIN user_books ub ON ub.book_id = w.book_id AND ub.status = 'available'  
      AND ub.user_id != :user_id  
WHERE w.user_id = :user_id  
GROUP BY b.id, b.title, a.full_name, b.genre, w.created_at  
ORDER BY available_count DESC, w.created_at DESC;
```

Листинг 3.5 – Расчёт рейтинга пользователя

```
SELECT u.id, u.username, u.rating AS stored_rating, ROUND(AVG(r.rating), 2)
      AS calculated_rating, COUNT(r.id) AS total_reviews, COUNT(e.id) AS
      total_exchanges
FROM users u
LEFT JOIN reviews r ON r.reviewee_id = u.id
LEFT JOIN exchanges e ON e.id = r.exchange_id
WHERE u.id = :user_id
GROUP BY u.id, u.username, u.rating;
```